Sessional Mid-Quiz **May 27, 2025**

BANGLADESH UNIVERSITY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

L-2/T-I CSE 208 Data Structures & Algorithms II

Time: 30 mins **Marks: 30**

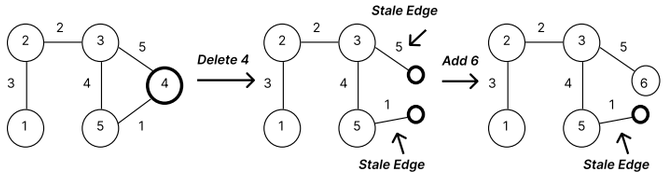
Student Name: _____ Student ID: _____

There are 5 questions in total. Answer all the questions in the spaces provided on the question sheets. No extra sheets will be provided

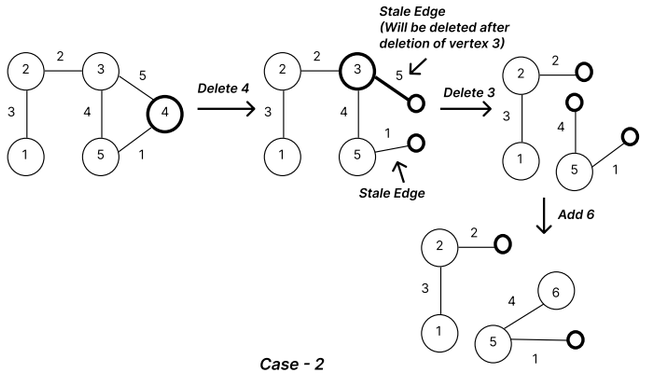
1. You are given a **weighted undirected graph** represented using an adjacency list. We will consider a *modified version of vertex deletion and addition* with the following rules: (6)

   - When a vertex is **removed**, all its incident edges are not deleted immediately. Instead: an edge becomes **stale** if exactly one of its endpoints is still present in the graph. An edge is **invalid** and must be deleted permanently if **both** of its endpoints have been removed.
   - When a **new vertex** is added to the graph, it will connect to the **stale edge with the maximum weight**, becoming the new endpoint of that edge. If there are no stale edges available, the new vertex remains unconnected.

   Write a short explanation (at most 5 sentences) describing your approach to handle stale edge creation during vertex removal, manage stale edge deletion when both endpoints are removed and reconnect stale edges when a new vertex is added. Then write pseudocode for the necessary functions.



Case - 1



Case - 2

Use a Boolean array to keep track of the edges that have been removed or not. When we are removing an edge, we will iterate over all its neighbor and push the weight and the neighbor in a max priority queue. Then when we are adding a new edge, we will extract one from the priority queue and then check that if the then active edge is now active or not.

```
Function remove_vertex(v):
    // Mark the vertex as removed
    is_deleted[v] = True

    // Iterate through all neighbors of the removed vertex
    For each (neighbor, weight) in adj_list[v]:
        // If the neighbor is still in the graph, the edge becomes stale
        If is_deleted[neighbor] == False:
            stale_heap.push( (weight, neighbor) )

Function add_vertex(new_v):
    // Initialize the new vertex in the graph
    is_deleted[new_v] = False

    // Try to find the maximum valid stale edge
    While stale_heap is not empty:
        weight, active_endpoint = stale_heap.pop()

        // Lazy Deletion: Check if the 'active' endpoint was deleted after this edge became stale
        If is_deleted[active_endpoint] == False:
            // Valid stale edge found; connect the new vertex to the active endpoint
            adj_list[new_v].append( (active_endpoint, weight) )
            adj_list[active_endpoint].append( (new_v, weight) )

            // Reconnection complete
            return

    // If the heap empties out, no valid stale edges exist, and new_v remains unconnected
```

2. You are a computer science undergraduate planning your academic journey. To graduate, you must complete a number of courses, but not all of them can be taken immediately - some require foundational knowledge from earlier courses. Each course takes a certain amount of time (in months) to complete. You can take multiple courses (maximum 3) in parallel if you have the required knowledge. (6)

You're particularly interested in completing **CS5** and **CS9**. You want to reach **CS5** and **CS9** as efficiently as possible, but you can't rush ahead without the required knowledge.

Here is your course roadmap:

Course	Prerequisites	Duration (Months)
CS1	-	3
CS2	-	3
CS3	CS1, CS2	6
CS4	CS3	3
CS5	CS4, CS7	2
CS6	CS2	3
CS7	CS2, CS3	5
CS8	CS3, CS6	2
CS9	CS4, CS8	6

Provide the order in which you will take the courses to efficiently complete:

(i) **CS5**

(ii) **CS9**

Also, determine the minimum total time (in months) required in each case to complete the course, including the necessary calculations.

(i) Efficiently Completing CS5

To reach CS5, you must complete its prerequisite tree: CS1, CS2, CS3, CS4, and CS7.

Schedule & Calculations:

- **Months 0 – 3:** Start with **CS1** (3 months) and **CS2** (3 months) in parallel.
  - *Time elapsed: 3 months.*
- **Months 3 – 9:** Both CS1 and CS2 are complete, unlocking **CS3** (6 months).
  - *Time elapsed: 3 + 6 = 9 months.*
- **Months 9 – 14:** CS3 is complete, unlocking both CS4 and CS7. You take **CS4** (3 months) and **CS7** (5 months) in parallel.
  - CS4 finishes at month 12.
  - CS7 finishes at month 14. You must wait for CS7 to finish before taking CS5.
  - *Time elapsed: 9 + 5 = 14 months.*
- **Months 14 – 16:** Both CS4 and CS7 are complete, unlocking **CS5** (2 months).
  - *Time elapsed: 14 + 2 = 16 months.*

Order of Courses:
(CS1, CS2) → CS3 → (CS4, CS7) → CS5

Minimum Total Time: 16 months

The algorithm to solve this question is very hard. I don't think it would come in my mind in 5 or 6 minutes. So I am not writing the algorithm here. This is done purely using brute force.

(ii) Efficiently Completing CS9

To reach CS9, you must complete its prerequisite tree: CS1, CS2, CS3, CS4, CS6, and CS8.

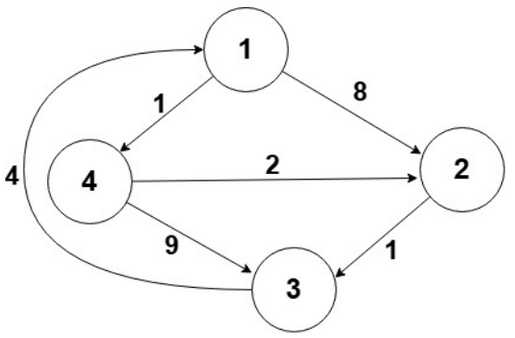
Schedule & Calculations:

- **Months 0 – 3:** Start with **CS1** (3 months) and **CS2** (3 months) in parallel.
  - *Time elapsed: 3 months.*
- **Months 3 – 9:** CS1 and CS2 are complete. CS2 unlocks CS6, and together they unlock CS3. You take **CS3** (6 months) and **CS6** (3 months) in parallel.
  - CS6 finishes at month 6, but CS8 also requires CS3, which is still ongoing.
  - CS3 finishes at month 9.
  - *Time elapsed: 3 + 6 = 9 months.*
- **Months 9 – 12:** CS3 and CS6 are now both complete. This unlocks CS4 and CS8. You take **CS4** (3 months) and **CS8** (2 months) in parallel.
  - CS8 finishes at month 11.
  - CS4 finishes at month 12. You must wait for CS4 to finish before taking CS9.
  - *Time elapsed: 9 + 3 = 12 months.*
- **Months 12 – 18:** Both CS4 and CS8 are complete, unlocking **CS9** (6 months).
  - *Time elapsed: 12 + 6 = 18 months.*

Order of Courses:
(CS1, CS2) → (CS3, CS6) → (CS4, CS8) → CS9

Minimum Total Time: 18 months

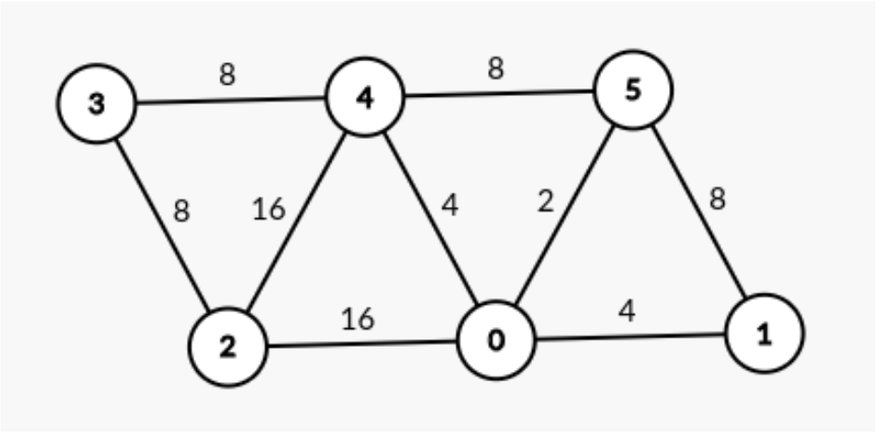
3. For the following graph, compute the distance matrices D_1, D_2, D_3, D_4 using the Floyd-Warshall algorithm. Here, D_k refers to the distance matrix after the $k-th$ iteration of the outermost loop (right after considering vertex k as the intermediate vertex). The initial matrix D_0 is provided as a reference. (6)



	1	2	3	4
1	0	8	∞	1
2	∞	0	1	∞
3	4	∞	0	∞
4	∞	2	9	0

Initial distance matrix, D_0

4. Given the graph, draw all possible spanning trees such that the product of the weights of the spanning tree edges is minimized. Mention the product in each case. (6)



5. Give an example of a directed graph (with at least 3 vertices) that satisfies the following conditions: (6)

- The graph has no negative weight cycles.
- The graph contains at least one edge with a negative weight.
- Dijkstra's algorithm fails to find the correct shortest path from a given source vertex.